ISOCODE SWE INTERN ASSESSMENT · BRIEF 4 · 9 NOVEMBER 2026

Kaapi Estate: *technical design*

A direct-booking website and an AI-routed enquiry inbox for a 76-key coffee-estate resort in Chikkamagaluru, Karnataka (64 rooms and 12 villas). “Kaapi Estate” is a working name; the brief calls the client “Company A”.

Code	github.com/SinghAbhinav04/kaapi
Live site	kaapi-estate.vercel.app
Staff console	kaapi-estate.vercel.app/staff/login (reviewer sign-in in `deployment.md`)
Author	Abhinav Singh

Throughout, `ASSUMPTION` marks anything not stated in the brief, and `BRIEF` marks a rule taken from it. Only facts from the problem statement are treated as facts.

---

1. The problem and what was built

`BRIEF` About 68% of room nights come through agencies at around 20% commission, so direct bookings are the goal. Enquiries arrive scattered across the web form, email, WhatsApp and an event form, and replies are slow. Agencies hold the rest of the 76 keys: the website may sell only its allotment, and must **never overbook**.

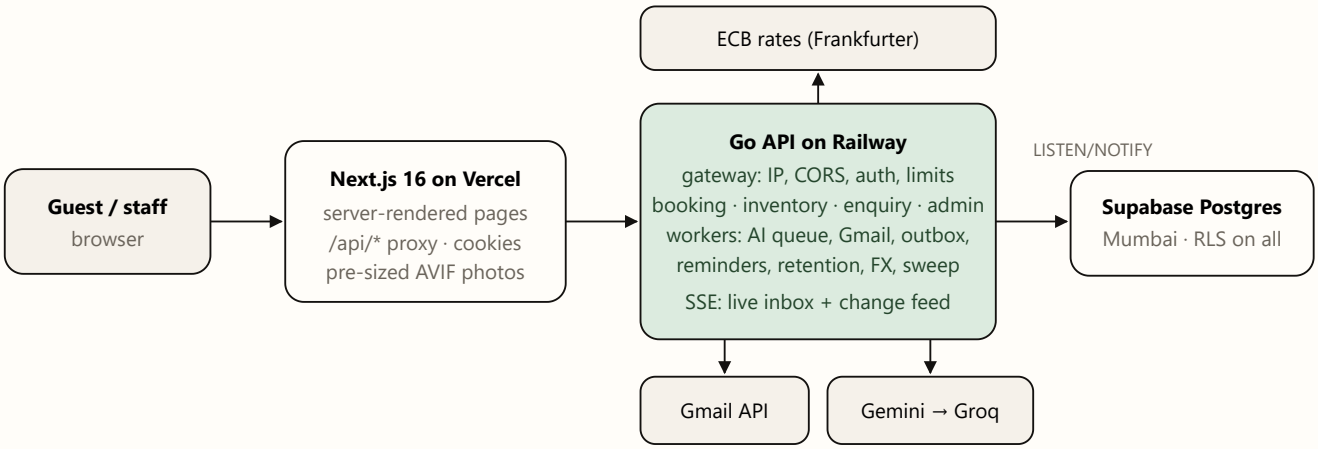
AREA	WHAT IT DOES
Guest site	Home with a CSS scroll-story hero; room pages with a nightly price calendar and GST per night; a 3-step booking (room → details → payment) with a 10-minute hold; a test payment gateway (UPI QR, card, net banking, wallet); confirmation; “My bookings” by email code; enquiry form; privacy page; prices shown as an estimate in the guest’s own currency (always charged in INR).
Booking engine	Holds that lock inventory, idempotent bookings, HMAC-signed payment webhooks, a late-payment refund path, background expiry, automatic festive offers and promo codes (the guest always gets the best one).
Enquiry inbox	Every channel lands in one inbox. AI classifies, extracts dates and party, routes to a lane and drafts a reply whose prices the server fills in. People edit and press Send; nothing is sent automatically.
Staff console	Today (what needs a reply now, the week’s arrivals and departures), Inbox, Rates & allotment heat map with bulk edits, Bookings (+ daily CSV), Outbox, Analytics, Offers, Team & access (database-backed roles).
Email	One connected Gmail mailbox: incoming mail becomes enquiries (guest replies join their thread), and replies, confirmations, reminders and opted-in offer emails go out from it, queued and retried.
Privacy	Personal data encrypted at rest (AES-256-GCM) with blind indexes for lookups, redaction before any AI call, DPDP consent and a retention schedule.

The brief's rules, and where each is enforced

RULE	HOW IT HOLDS
`BRIEF` Never overbook	Three layers: `FOR UPDATE` row locks in date order, live hold counting, and a database `CHECK (sold <= allotment)`. A test races 25 buyers for the last villa: exactly one wins.
Prices come from the rate table, never the AI	The model writes wording with a `{{QUOTE}}` placeholder; the server fills it from the same search the website uses. Any money the model writes is deleted and flagged.
Drafts are never auto-sent	Only `POST /v1/staff/enquiries/{ref}/send`, pressed by a person, sends a reply.
Unsure enquiries go to Unsorted; re-routing is one click	Confidence below 0.6 → the Unsorted lane, which the GM sees. Re-routing is one optimistic click and is logged.
Complaints always alert the GM	If either the model or the keyword rules sees a complaint (main or secondary category), the enquiry goes to the GM lane at high priority, the console's bell alerts, and fresh complaints are emailed to every GM and administrator, once per enquiry.
Booking in 3 steps or fewer, fast on slow mobile	Room → details → payment. Pages are server-rendered with data; photos are pre-sized AVIF (480/960/1600 px) chosen per device.

---

2. Architecture



- **One Go service plus in-process workers.** The API (chi, pgx, hand-written SQL, goose migrations embedded in the binary) also runs the background jobs; `cmd/worker` can run them separately. A gateway layer inside the API (trusted client IP, CORS and rate limits per route group, live-session token checks, CSRF by `Origin`) was chosen over a separate gateway service: one service doesn't earn a second hop or trusting identity headers between processes.
- **Contract-first.** `contract/openapi.yaml` is written by hand. The integration tests validate *every* real response against it, and the web app's types are generated from it, so the three can't drift.
- **Package layout.** Each domain package has `service.go` (rules and transaction boundaries), `store.go` (every query), types and errors. Money is `int64` paise; stay dates are IST calendar dates; IDs are UUIDv7;

booking refs are `KS-XXXXXX`.

- **Frontend.** Next.js 16 App Router with React 19 (React Compiler on), TanStack Query, CSS Modules with design tokens, no UI library. Server components prefetch data and hydrate the client cache; filtered views keep the previous data on screen while new data loads.
- **Live data without polling.** Statement-level triggers send a topic through `pg_notify` on changes to bookings, inventory, rates, holds, offers and enquiries. The API streams topics to staff consoles over server-sent events, and each page refetches only what the topic affects. The same feed clears the analytics cache.

---

3. The booking engine: never overbook

1. `SELECT … FROM night_inventory … ORDER BY date FOR UPDATE`: locking nights in date order means two overlapping holds can't deadlock.
2. Check stop-sell, minimum stay, closed-to-arrival and closed-to-departure.
3. Count live holds (`status = 'ACTIVE' AND expires_at > now()`), so expired holds stop counting with no cron job.
4. Require `sold + held + rooms ≤ allotment` for every night; insert the hold with a frozen quote and a hashed token.
5. A verified payment webhook (HMAC, idempotent per event id) re-locks, increments `sold`, writes the booking's nights and confirms. A payment that arrives after the room went takes the `REFUND_PENDING` path, alerts the GM and tells the guest; later events for that order change nothing.
6. Last line of defence: `CHECK (sold <= allotment)` in the database.

Pricing is a pure package (no I/O): nightly price by plan, offers (percent in basis points capped at 50%, or flat), and GST per night. The search and the hold use the same function, and a test keeps their numbers equal. A search runs seven queries however many rooms and plans there are.

`ASSUMPTION` GST is shown per night at the tariff slab: 5% up to ₹7,500 a night, 18% above, on the price after discount. `ASSUMPTION` The hold lasts 10 minutes (a property setting). `ASSUMPTION` The roughly 20% commission saved is an estimate from the brief's figure, used in analytics.

---

4. The AI enquiry pipeline

1. **Intake** from any channel stores the enquiry as pending; duplicates per sender per day are merged, with at most 5 per sender per hour from the web form.
2. **Redaction** removes emails, phone numbers (10+ digits) and the sender's name, in any script, before any model sees the text.
3. **Classification** runs a model chain (fast Gemini models round-robin, then Gemini Pro, then Groq), each paced to its rate limit and rested after errors. A keyword classifier (English, Hinglish and Kannada-English) always runs too: it answers when every model is down and acts as the complaint safety net.
4. **Routing** is a rule table, not a model decision: complaint → GM (high priority, alert); confidence < 0.6 → Unsorted; room → Reservations; wedding or offsite → Sales. A person's re-route is never undone by a retry.

5. **Drafting:** the model writes wording only, with placeholders. The server fills the quote from live availability (saying what it assumed when the guest didn't give a party), and signs the reply as whoever presses Send.

59/60	**60/60**	**0**	**11/11**
categories correct (model chain + rules)	routed to the right team	sent to the wrong team	complaints reached the GM

From `go run ./cmd/eval` on 60 hand-labelled enquiries (including the brief's examples, Hinglish, Kannada-English, a politely disguised complaint, mixed intent and spam); full report in `docs/ai-eval-report.md`. Keyword rules alone: 57/60, 3 to the wrong team, 10/11 complaints. A confidence blend with the keyword rules was removed because the evaluation showed it changed no routing.

---

5. Data

TABLES	PURPOSE
`room_types`, `rate_plans`, `night_rates`, `night_inventory`	Catalogue; price per plan per night; the website's allotment, sold count and restrictions per room type per night.
`holds`, `bookings`, `booking_nights`, `payments`, `payment_events`	The booking flow; events are stored once per provider event id (idempotency).
`enquiries`, `enquiry_drafts`, `enquiry_events`, `enquiry_messages`	The inbox: classification, versioned drafts with a send claim, an audit trail, and guest follow-ups in a thread.
`outbox_messages`, `notifications`, `campaigns`, `booking_reminders`	Every message sent or simulated (queued, retried, never lost); console alerts; offer emails; reminders sent exactly once.
`users`, `roles`, `permissions`, `sessions`, `otp_codes`, `audit_logs`	Accounts, database-backed access control, rotating sessions, one-time codes, and an audit of every administrative change.

19 migrations; every table has row-level security, so Supabase's public key exposes nothing (a test fails if a table lacks it). Hot queries were checked with `EXPLAIN ANALYZE` at production scale and indexed (inbox, analytics covering index, thread lookups).

6. Security and privacy

- **Sessions:** a 15-minute encrypted access token (JWE) plus a rotating refresh token stored hashed; reusing an old refresh token revokes the whole session chain. Every request also checks the session is live, so signing out or deactivating someone takes effect at once. Browser tabs refresh one at a time, so they can't trip the reuse detector.
- **Staff sign-in** is a password plus an emailed 6-digit code (5 minutes, 5 attempts). `TRADE-OFF` One reviewer account signs in with the password alone (`STAFF_PASSWORD_ONLY`), because reviewers don't have its mailbox; an empty setting turns this off.
- **Guests** sign in with an emailed code that is never shown on screen or returned by the API. Booking requires a verified email, so a typo can't lose the confirmation.
- **Access control:** permissions are a closed set in code, granted per role in the database and checked on every route. Only an Administrator can grant the Administrator role or change an Administrator's account; nobody can grant permissions they don't hold; the last Administrator can't be removed.
- **Gateway:** client IPs from `X-Forwarded-For` counted from the right (a forged entry is ignored); CORS per route group; rate limits per group (sign-in, codes, holds, bookings, enquiries shared across instances in Postgres; the rest in memory). `ASSUMPTION` The limits are sized for one resort (e.g. staff login 10 per 15 minutes per IP).
- **Personal data encrypted at rest:** names, emails, phones and outbox copies use AES-256-GCM through a query-argument interceptor (`pii.Text`), so domain code only sees plain strings. Lookups use keyed blind indexes (HMAC). `ASSUMPTION` Enquiry text stays plaintext on purpose: the AI needs it (after redaction) and inbox search uses it.
- **DPDP:** consent at checkout, minimal fields, no passport details online, redaction before any AI call, and offer emails only with opt-in and a one-click unsubscribe.
- **Retention** (a job at start-up and every 6 hours, audit-logged): archive finished bookings after check-out; anonymise contact details 12 months after the stay; delete bookings once 8 full financial years have passed; delete enquiries, outbox copies and alerts at 12 months. `ASSUMPTION` 8 financial years follows Companies Act s.128, which outlasts GST's 72 months.
- **Mail safety:** mail to reserved test domains is never handed to Gmail; the poller skips bounces and auto-replies.

7. Frontend and performance

- Design tokens (paper, ink, one estate-green accent, cherry for alerts), Anton / Instrument Serif / Manrope / Space Mono. Motion uses only transform and opacity and switches off under `prefers-reduced-motion` .
- The hero is a pinned scroll story in pure CSS (`animation-timeline`): no JavaScript runs while scrolling, nothing repaints per frame (no SVG filters on moving things).
- Photos are AVIF in pre-built sizes chosen by a custom image loader. Vercel's optimizer had been passing AVIF through at full size, so phones downloaded 2400-pixel photos. On the mobile home page (Lighthouse, slow 4G simulation), this took Performance from 75 to 85, the largest paint from 5.7 s to 4.3 s, and page weight from 983 KB to 548 KB. Desktop home: 95. Privacy page on mobile: 98. Accessibility: 100.

- Accessibility: 44 px tap targets, labelled controls, errors linked with `aria-describedby`, correct alert and status roles; charts have a table view.
- `Server-Timing` on every response (except sign-in routes, where a query count would reveal whether an account exists) reports database time and query count, so latency is read in the browser rather than guessed.

---

8. Quality

- **Tests:** unit tests for pricing, redaction, the keyword classifier, tokens and money; integration tests against a real Postgres (embedded locally) covering the booking race, holds, payments and refunds, routing, Gmail (against a fake Google server), access control, retention, encryption at rest and the gateway. Every API response in the tests is validated against the OpenAPI contract.
- **Checks before merging:** `go vet`, `gofmt`, the full test suite; `tsc`, `eslint --max-warnings=0` and `next build` for the web app.
- **Review on 9 November:** three review passes and a browser/performance pass found 63 issues. All the serious ones are fixed with tests (for example: a payment event could overwrite a refund in progress; a complaint mentioned in a booking request didn't reach the GM; non-Latin names reached the model; a team manager could make themselves Administrator; the Gmail import could stall). The rest are fixed or recorded with a reason in `docs/notes/code-review-2026-11-09.md`.

9. Assumptions

ASSUMPTION	WHY
Working name “Kaapi Estate”	The brief says only “Company A”.
GST per night: 5% to ₹7,500, 18% above	The tariff slabs for hotel rooms, applied to the price after discount.
~20% agency commission saved per direct booking	The brief’s figure, used to estimate commission saved in analytics.
10-minute room hold	Long enough to pay on a slow phone, short enough not to block inventory.
First-reply target: 30 minutes	The success measure the Today page and the sidebar track.
Unsorted below 0.6 confidence	Chosen from the evaluation’s threshold sweep (no wrong-team routings at 0.5–0.8).
Retention: 12 months for enquiries and contact details; 8 financial years for bookings	DPDP data minimisation against tax and company-law record keeping.
Rate limits (e.g. 10 logins / 15 min per IP, 20 holds / 10 min)	Sized for one resort’s traffic.
Enquiry text stored in plaintext	Needed for AI classification (after redaction) and inbox search.
Gateway inside the API	One service doesn’t justify a second hop; split out when a second service exists.
Password-only reviewer account	Reviewers can’t receive its sign-in codes.
Reminder timings (1–3 days before, 8 am on arrival day, …)	Common hotel practice.
Placeholder imagery, UPI merchant and estate map pin	Until the resort supplies its own.
Exchange-rate estimates from ECB rates, shown only while under 48 hours old	Guests read prices in their own currency but are always charged in INR.

10. Trade-offs and what’s next

- **Real payment gateway.** The provider boundary (signed events, idempotency, late-payment refunds) is built; a gateway such as Razorpay plugs in behind it.
- **WhatsApp** replies are recorded as simulated until a WhatsApp Business API account exists; email is live.
- **Scale-out.** Analytics cache and some rate-limit counters are per process; Redis (behind the existing interfaces) is the step when several API instances must share them. Gmail push (Pub/Sub) can replace the 5-second poll.
- **Guest data requests.** “Delete my data” requests (OTP-verified, a staff queue, export) are designed but not built.

- **Sessions** renew for 30 days on use, with no absolute cap yet; an administrator cap (e.g. 12 hours) is a small change.
- **Mobile app.** The API already issues Bearer tokens to native clients (`X-Client: native`); a Capacitor shell over the site is the planned app.